

Simulazione di uno stormo inseguito da un predatore

Mila Casali, Francesco Fonseca

1 luglio 2025

Modifiche intercorse dall'ultima consegna

Di seguito sono elencate le modifiche intercorse dall'ultima consegna:

- La struttura del progetto è stata riorganizzata in sette translation unit e sette file di intestazione.
- La classe Vec3, è stata convertita in una struct, in quanto contenitore di soli dati e funzioni strettamente correlate, e senza necessità di proteggere gli attributi.
- La classe Boid è stata trasformata in una classe base astratta dotata di alcune funzioni puramente virtuali. La simulazione è ora progettata per operare esclusivamente su oggetti delle classi derivate, ovvero Predator e Prey.
- Alcuni cicli for tradizionali sono stati sostituiti con l'uso di algoritmi della libreria standard C++ (std::for_each, std::remove_if).
- Le statistiche della simulazione non vengono più stampate a schermo, ma salvate su un file, permettendo un'analisi quantitativa successiva.
- I parametri sono adesso gestiti da tre strutture globali.
- I valori limite della simulazione sono stati ristretti per garantire un comportamento realistico anche ai casi limite.
- Le unità di misura dei parametri sono state modificate.
- È stata aggiunta la possibilità di specificare un angolo di visione per i boid di tipo Prey, limitando l'interazione ai soli vicini effettivamente visibili.
- È stato implementato un comportamento più verosimile nella fase iniziale: i boid partono dalla base dello spazio simulato e si alzano in volo all'avvio della simulazione.
- È stata implementata una funzione di stallo che, in caso di assenza di movimento, simula una caduta.
- A livello grafico, è stato modificato il colore dello sfondo delle finestre: il piano XY, ora di colore verde, rappresenta una prateria, mentre il piano XZ, di colore blu, simula visivamente il cielo.

Introduzione

Gli stormi di volatili sono in grado di spostarsi in modo coordinato nella stessa direzione, senza la presenza di un capo o di una figura di autorità che li guidi o organizzi. Dunque il nostro scopo è riuscire a simulare uno stormo che si comporti in modo ordinato e coeso partendo dall'implementazione del moto di un singolo volatile. Il problema di riuscire ad emulare tale comportamento venne affrontato da Craig Reynolds nel 1986 [1], che propose il modello al quale ci rifacciamo in questo progetto. Tale modello prende il nome dal software ideato da Reynolds che venne battezzato "boids".

1. Analisi del problema

La simulazione avviene in uno spazio tridimensionale toroidale e si basa sul fatto che ogni singolo ente segua tre regole fondamentali ideate dallo stesso Reynolds [2]: la separazione, che fa in modo che ogni uccello eviti di avvicinarsi eccessivamente ai vicini, l'allineamento, che spinge ogni volatile ad adattare la propria velocità a quella dei compagni e infine la coesione, che mantiene l'unità dello stormo facendo in modo che ogni uccello vada verso il centro dello stesso. In aggiunta a queste tre

regole basilari abbiamo implementato la simulazione in modo tale che tenga conto della presenza di un predatore dal quale gli uccelli fuggono e in modo tale che il moto dello stormo sia influenzato dalla presenza del vento che spira in una specifica direzione. Inoltre gli uccelli partono dalla base dello spazio simulato e si alzano in volo all'avvio della simulazione, mantenendo poi una quota predeterminata. Ognuna di queste indicazioni è coadiuvata da fattori limitanti sulla velocità e posizione dei singoli volatili in modo tale da avere un comportamento realistico, tali fattori potranno essere inseriti dall'utente o generati casualmente, essi includono: l'apertura alare, la velocità massima, la distanza minima, l'angolo visivo, i fattori di allineamento, coesione, separazione e paura del predatore, la velocità del vento, la velocità e il raggio di attacco del predatore. Inoltre saranno sempre a discrezione dell'utente la presenza del vento e dello spazio toroidale.

2. Progettazione dell'Algoritmo

2.1 Scelte di Progetto

Come ausilio allo sviluppo del programma abbiamo utilizzato la repository gitHub riportata in appendice [3], nella quale sono state caricate gradualmente la maggior parte delle versioni del codice. Per quanto riguarda l'ambiente in cui si muovono i boids abbiamo scelto di implementare 2 possibili strategie: un ambiente con confini rigidi che determinano un "rimbalzo" dei boid e uno spazio toroidale per far sì che i volatili si muovano senza confini rigidi. La scelta del tipo di ambiente è effettuata dall'utente.

Relativamente alla simulazione grafica del volo, abbiamo utilizzato la libreria grafica SFML per gestire la visualizzazione della simulazione proiettata in 2D su 2 piani (XY e XZ).

2.2 Passi Principali dell'Algoritmo

L'algoritmo che regola il movimento dello stormo consiste nei seguenti passi:

1. Inizializzazione dello stormo: generazione delle posizioni e delle velocità iniziali dei boid.
2. Calcolo delle forze: per ogni boid, in base alle posizioni e alle velocità dei boids vicini, calcolo delle forze di separazione, allineamento e coesione.
3. Influenza ambientale: applicazione delle forze derivanti dal vento e dalla presenza del predatore.
4. Aggiornamento del movimento: aggiornamento delle velocità e delle posizioni dei boid.
5. Gestione dei bordi: applicazione delle regole toroidali o di rimbalzo ai confini dello spazio di simulazione.
6. Interazione con il predatore: il predatore cerca di catturare i boid, che a loro volta tentano di fuggire.
7. Estrazione delle statistiche: ad ogni unità di tempo (o multipli) vengono estratte, e salvate su file, una serie di misure che riassumono il comportamento dello stormo.

3. Implementazione

Il progetto è organizzato modularmente, suddiviso in sette translation unit e altrettanti file di intestazione che contengono le definizioni delle classi e le funzioni ausiliarie. Il file principale, 'boids.cpp', funge da entry point per l'intero programma e coordina l'interazione tra le varie classi e funzioni, gestisce l'inizializzazione dei parametri della simulazione, il salvataggio delle statistiche e l'aggiornamento in tempo reale delle finestre grafiche.

Oltre al file principale, ci sono alcuni file di libreria che contengono le classi fondamentali per il funzionamento del programma. Questi file sono:

- **vec3:** Contiene la definizione della struct 'Vec3', che rappresenta i vettori tridimensionali utilizzati per gestire le posizioni e le velocità dei boids nello spazio. Questa classe include operazioni vettoriali essenziali per il movimento e l'interazione dei boids.
- **boid:** Contiene la definizione della classe madre 'Boid', l'unità di base della simulazione, che rappresenta i singoli volatili. Ogni boid ha come proprietà una posizione e una velocità. Sono inoltre presenti metodi volti all'aggiornamento delle velocità e delle posizioni dei boids.
- **prey:** Definisce la classe 'Prey', derivata da 'Boid' e che ne completa l'implementazione per rappresentare un volatile membro di uno stormo.
- **predator:** Definisce la classe 'Predator', una specializzazione di 'boid' che introduce un predatore nello stormo. Questo file gestisce le caratteristiche uniche del predatore, come la capacità di cacciare i boids, aggiungendo un ulteriore livello di complessità alla simulazione.
- **swarm:** Definisce la classe 'Swarm', che gestisce lo stormo di boids. Questa classe è responsabile di aggiornare lo stato di ogni boid e di regolare le interazioni tra di essi, oltre a gestire la logica globale dello stormo.

I File di Utility e le Variabili di Configurazione sono inclusi nelle seguenti librerie:

- **functions:** Include funzioni ausiliarie e utility che supportano la logica principale del programma. Queste funzioni possono gestire l'interfaccia utente, inizializzazione dei parametri, calcoli matematici o altre operazioni necessarie per il corretto funzionamento della simulazione.
- **variables:** Definisce e inizializza le strutture dati che contengono i parametri di configurazione globali per la simulazione.

Il testing della libreria è realizzato tramite il seguente file:

- **boids.test.cpp:** Questo file contiene l'implementazione dei test numerici per verificare il corretto funzionamento delle principali classi e funzioni del progetto. Utilizza la libreria Doctest.

Intelligenza Artificiale Come ausilio all'implementazione sono stati usati software di intelligenza artificiale generativa. L'uso è stato principalmente volto all'implementazione delle funzioni grafiche, oltre che per velocizzare il processo di debugging.

3.1 Descrizione dettagliata delle classi fondamentali

3.1.1 La struct "Vec3"

La struct 'Vec3' rappresenta un vettore tridimensionale e implementa vari metodi per operazioni vettoriali di base: somma, sottrazione, prodotto per uno scalare, prodotto scalare e vettoriale, normalizzazione e calcolo della norma del vettore. Inoltre, la struct include alcune operazioni necessarie al calcolo delle distanze in uno spazio toroidale. La struct include anche un metodo per accedere agli elementi del vettore tramite l'overload dell'operatore '[' e per stampare il vettore tramite l'operatore '<<'.

3.1.2 Gestione globale dei parametri

Le strutture 'SwarmVariables', 'PredatorVariables' e 'GlobalVariables' contengono i parametri relativi allo stormo, al predatore e quelli condivisi. I costruttori delle classi 'Swarm' e 'Predator', descritte in seguito, accettano infatti oggetti di tipo 'GlobalVariables' e 'SwarmVariables' per lo stormo e 'GlobalVariables' e 'PredatorVariables' per il predatore. Questo serve a minimizzare la possibilità di avere valori diversi per i parametri condivisi.

Le strutture ‘ConversionFactors’ e ‘LimitValues’ contengono invece fattori di conversione e parametri utilizzati nell’interfaccia utente.

3.1.2 La classe “Boid” e sottoclassi “Pray” e “Predator”

La classe Boid è la classe base astratta del sistema di simulazione, progettata per rappresentare qualsiasi entità mobile nello spazio tridimensionale. Essa definisce una struttura comune per tutti gli agenti, fornendo come attributi fondamentali la posizione e la velocità (entrambi rappresentati tramite oggetti di tipo Vec3), oltre a metodi per leggerli, modificarli e confrontare oggetti. Include inoltre le operazioni di aggiornamento (updateBoidVelocity, updateBoid) e di gestione dei bordi (border). La presenza di metodi puramente virtuali come stall e resetCooldown ne impedisce l’istanziatura diretta, obbligando le classi derivate a fornire implementazioni concrete.

Particolarmente importante è la funzione ‘border’. Nel caso in cui lo spazio toroidale sia attivo, se un boid supera i limiti orizzontali (assi x e y), viene spostato all’interno dell’area di simulazione, ricomparendo sul lato opposto. Per quanto riguarda l’asse verticale (z), la funzione prevede un comportamento di rimbalzo parzialmente anelastico.

Nel caso in cui non sia attivo lo spazio toroidale, qualsiasi incontro con i bordi viene trattato come un urto parzialmente anelastico.

La classe Prey estende Boid e rappresenta i volatili ordinari dello stormo. Implementa il metodo stall, che simula una caduta se la velocità risulta troppo bassa rispetto al vento, e ridefinisce resetCooldown come funzione vuota, in quanto le prede non hanno tempi di attesa tra le azioni.

La classe Predator è anch’essa una sottoclasse concreta di Boid, ma dotata di un comportamento finalizzato alla caccia. Aggiunge attributi specifici: attack_range, attack_speed, preferred_height e un contatore cooldown per regolare le fasi di attacco e riposo. I metodi principali includono findPrey, che individua la preda più vicina, attack, che modifica la direzione del predatore per inseguirla, e stall, che simula una discesa quando la velocità si abbassa sotto una soglia, inferiore rispetto a quella delle prede. Il comportamento del predatore alterna fasi attive e passive, gestite da un tempo di cooldown che viene azzerato dalla funzione ‘resetCooldown’ dopo ogni cattura.

3.1.3 La classe “Swarm”

La classe ‘Swarm’ è il fulcro del progetto, essa rappresenta uno stormo di preys e ne gestisce la dinamica di movimento attraverso il suo metodo principale ‘updateSwarm’, che aggiorna posizione e velocità di ogni boid tenendo conto delle regole comportamentali, della presenza di un predatore, e delle condizioni ambientali.

La posizione iniziale dei boid viene generata casualmente alla base della simulazione entro i limiti dello spazio definito da ‘screen’. La velocità è generata casualmente ma limitata da ‘max_speed’: se il modulo del vettore velocità supera ‘max_speed’ il vettore viene normalizzato, così da mantenere la direzione iniziale ma imponendo la velocità massima come modulo.

Le regole comportamentali sono:

coesione. La funzione ‘cohesion’ implementa la regola che tende a tirare ogni boid verso il centro di massa percepito dello stormo. L’obiettivo è avvicinare ogni boid agli altri per mantenere la coesione del gruppo.

La funzione scorre tutti i boid nello stormo, calcolando la posizione media di quelli che si trovano all’interno di una certa distanza (‘sight_distance’) e un determinato angolo (‘visual_field’) rispetto al boid attualmente considerato. Se non ci sono boid vicini, la

funzione restituisce un vettore nullo, altrimenti un'accelerazione dipendente dalla posizione del centro di massa e scalata da un fattore 'cohesion_factor'.

separazione. La funzione 'separation' implementa la regola che ha l'obiettivo di mantenere una distanza minima tra ciascun boid e i suoi vicini, evitando collisioni. Se un boid si trova a una distanza inferiore o uguale alla distanza minima e all'interno del campo visivo, viene calcolato un vettore correttivo diretto lontano dal volatile e inversamente proporzionale alla distanza tra i due boids, scalato da un fattore 'separation_factor'.

La funzione 'bounce' gestisce il comportamento dei boids in caso di collisione, trattando gli scontri come urti parzialmente anelastici.

allineamento. La funzione 'alignment' implementa la regola che permette a ciascun boid di allineare la propria velocità a quella dei vicini. La funzione restituisce la media delle velocità dei boid vicini e all'interno del campo visivo, scalata da un fattore 'alignment_factor'.

altezza preferenziale. La funzione 'height', fa sì che ogni boid dello stormo applichi delle correzioni al proprio movimento per mantenersi a un'altezza preferenziale ($\frac{2}{3}$ dello schermo come impostazione di default). In particolare, se il boid si trova a un'altitudine diversa da quella predefinita e la sua velocità verticale lo porterebbe ad allontanarsi da tale quota, viene applicata una correzione proporzionale alla sua distanza dall'altitudine preferenziale e scalata mediante un fattore 'height_factor'.

paura. la funzione 'fear', attiva nel momento in cui viene impostata la presenza di un predatore, gestisce il tentativo di evasione dei volatili da esso. Se questi si trova all'interno della distanza di visuale la funzione restituisce una velocità di evasione che ha come direzione quella della distanza fra boid e predatore e con modulo pari alla velocità massima del boid a cui viene sottratto un fattore pari al rapporto fra la distanza predatore-boid e la distanza di visuale del boid, il tutto scalato per un fattore 'fear_factor'.

Nel metodo 'updateSwarm', i boid catturati dal predatore vengono eliminati dallo stormo, mentre le velocità e le posizioni degli altri vengono aggiornate tenendo conto delle regole comportamentali, del vento e infine dei bordi.

3.2 Le funzioni ausiliarie

Nel file 'functions.cpp' sono definite alcune funzioni di inizializzazione dei parametri dello stormo e aggiornamento della simulazione

3.2.1 Inizializzazione dei parametri

La funzione 'initializeParameters' è la principale responsabile della gestione dell'interfaccia utente. Questa funzione consente di stabilire i parametri di base che determinano il comportamento dello stormo e le condizioni ambientali in cui esso si muove. In particolare, l'utente ha la possibilità di abilitare o disabilitare la presenza del vento, di uno spazio toroidale, e del predatore. La funzione supporta sia un'inizializzazione automatica dei parametri, con valori predefiniti, sia un'inizializzazione manuale, dove l'utente può specificare parametri all'interno di range predefiniti. La funzione offre anche la possibilità di generare i parametri in modo casuale richiamando 'casualBoidsParameters' e 'casualSimulationParameters'. La prima gestisce la generazione di tutti i parametri ad esclusione dei fattori di modulazione delle regole comportamentali, generati dalla seconda.

3.2.2 Calcolo e salvataggio delle statistiche

La funzione 'saveParametersOnFile' è responsabile del salvataggio dei parametri iniziali della simulazione su un file (specificato da filename). Include anche le intestazioni per i dati statistici che verranno aggiunti successivamente da printStatistics.

La funzione statica 'printStatistics' calcola e stampa le statistiche correnti dello stormo (distanza media, deviazione standard della distanza, velocità media, deviazione standard della velocità e numero di boid) su un file. Se lo stormo contiene un solo boid, imposta la distanza media a zero per evitare di interrompere la simulazione. La funzione è richiamata periodicamente da 'updateSimulation'.

3.2.3 Aggiornamento della simulazione

La funzione 'updateSimulation' si occupa di aggiornare le posizioni e le velocità sia del predatore sia dello stormo richiamando le relative funzioni di update. Inoltre, questa funzione, chiamando 'printStatistics', esegue un salvataggio periodico su file delle statistiche.

3.2.4 Gestione dell'Aspetto Grafico dei Boid

La rappresentazione visiva della simulazione è gestita principalmente attraverso le funzioni 'drawWindows', 'drawBoids', e 'handleEvents'. Queste funzioni, che sfruttano la libreria SFML, permettono di creare, gestire e aggiornare le finestre grafiche in cui viene visualizzata la simulazione.

La funzione 'drawWindows' è responsabile della creazione delle finestre di visualizzazione. Utilizzando la risoluzione del desktop dell'utente ('getDesktopMode'), questa funzione calcola le dimensioni appropriate per le finestre, tenendo conto delle dimensioni dello schermo e della barra delle applicazioni. Due finestre vengono create per visualizzare il volo dei boid da diverse prospettive: la Top View, con sfondo verde per rappresentare un prato, e la Side View, di colore azzurro a rappresentazione del cielo.

La funzione 'drawBoids' si occupa del disegno effettivo dei boids e del predatore all'interno delle finestre create. I boid vengono rappresentati come cerchi neri, con un raggio corrispondente all'apertura alare, mentre il predatore, più grande, è disegnato come un cerchio rosso per distinguerlo chiaramente dal resto dello stormo. La posizione dei boids e del predatore all'interno delle finestre è calcolata in base al rapporto tra le dimensioni dello spazio di simulazione e delle finestre.

La funzione 'handleEvents' gestisce l'interazione tra l'utente e le finestre della simulazione. Utilizzando un loop di eventi, questa funzione verifica se l'utente ha chiuso una delle finestre e, in tal caso, interrompe il programma.

3.3 Il file main (boids.cpp) e l'interazione con l'utente

Il file boids.cpp gestisce l'interazione tra utente, Swarm e Predator. Dopo aver chiamato le necessarie funzioni di inizializzazione, definite in 'functions.cpp', il main disegna le finestre e avvia la simulazione. Questa viene aggiornata mediante un ciclo while, attivo fino a quando sono presenti boids nello stormo ed entrambe le finestre sono aperte. Abbiamo inoltre impostato un valore massimo di 60 frame al secondo per evitare che la simulazione acceleri eccessivamente quando sono presenti pochi boids.

4. Testing del Programma

Per verificare il corretto funzionamento del programma sviluppato, sono stati eseguiti diversi test sulle funzioni numeriche utilizzando la libreria Doctest. I test sono stati progettati per coprire le principali funzionalità delle classi e delle funzioni implementate, assicurandosi che il comportamento sia quello previsto.

Inoltre, il funzionamento del simulatore, non essendo facilmente descrivibile matematicamente, è stato testato variando i parametri della simulazione e acquisendo le statistiche di output. Se l'implementazione è corretta, ci si aspetta che la distanza media e la deviazione standard delle velocità tendano a ridursi nel tempo.

Se presente, gli attacchi del predatore dovrebbero essere accompagnati da un aumento di distanza e velocità media. Il vento dovrebbe rallentare il processo di compattazione e velocizzare l'uniformazione delle velocità, mentre la presenza di un bordo solido dovrebbe avere l'effetto opposto.

Simulazione al variare dei parametri

In questa sezione presentiamo alcuni risultati ottenuti variando i parametri della simulazione. Mantenendo costanti i seguenti parametri:

Apertura alare: 0.3 m

Distanza minima: 2 m

Raggio visivo: 15 m

Angolo visivo: 240 °

Velocità massima: 6 m/s

Per ciascun test sono riportati gli screenshot della simulazione e dei grafici che riportano velocità e posizione dei boid nel tempo e le rispettive deviazioni standard, questi ultimi realizzati su python con l'ausilio dell'IA. (Tali grafici non fanno parte del progetto, sono ausili all'analisi dei dati).

4.1 Test 1: Simulazione senza vento e con bordi

Input:

- Spazio toroidale: Disattivato
- Vento: Assente
- Numero di boid: 120
- Range di attacco: 70 m
- Velocità di attacco: 9 m/s
- Parametri di separazione, allineamento, coesione, paura e altezza: 90%, 90%, 90%, 90%, 90%

Output:

Stormo che pian piano diventa coeso e poi si muove lentamente nello spazio tridimensionale senza disgregarsi eccessivamente. Nel momento in cui il predatore attacca lo stormo scappa e rimbalza sui bordi. Nella rappresentazione grafica delle statistiche di simulazione [Fig 2] sono evidenti i minimi di velocità dovuti alle collisioni coi bordi e massimi dovuti alla fuga dal predatore.

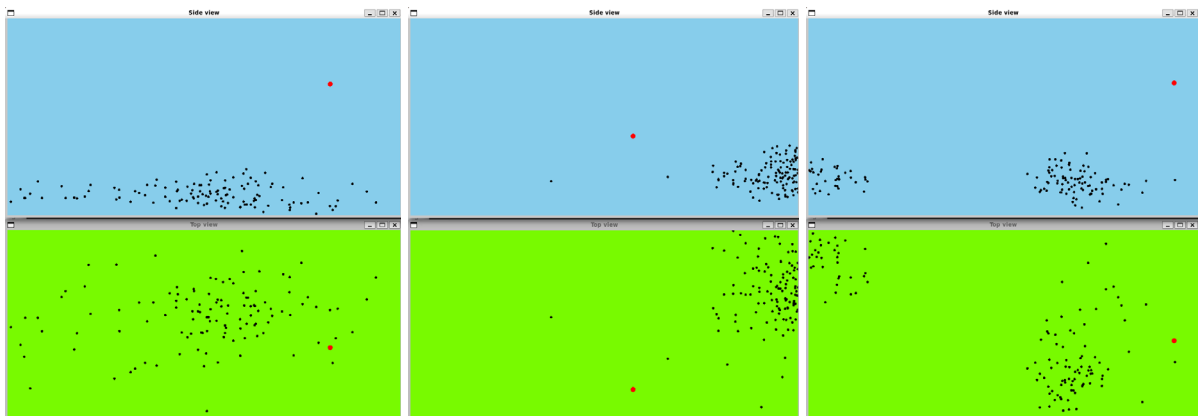


Figura 1: Screenshot della simulazione 1

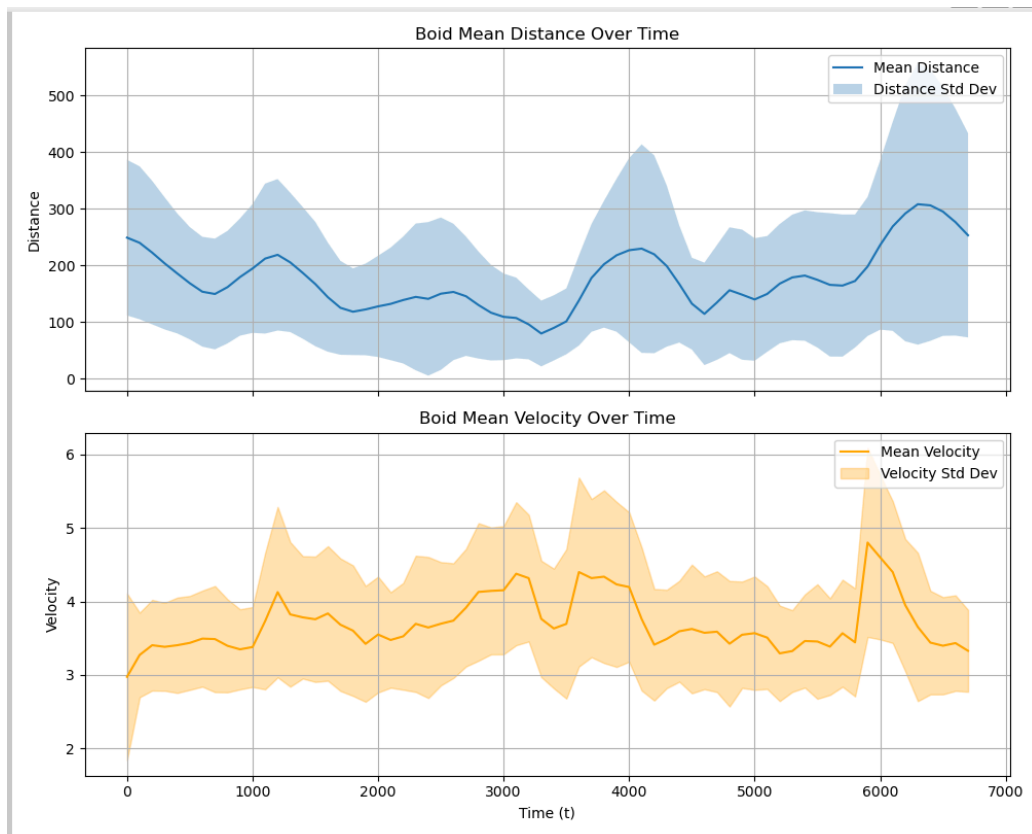


Figura 2: statistiche della simulazione 1

4.6.2 Test 2: Simulazione senza vento e con spazio toroidale

Input:

- Spazio toroidale: Presente
- Vento: Assente
- Numero di boid: 90
- Range di attacco: 30 m
- Velocità di attacco: 7.8 m/s
- Parametri di separazione, allineamento, coesione, paura e altezza: 50%, 50%, 50%, 50%, 50%

Output:

Lo stormo è diradato e si muove nello spazio toroidale fuggendo dal predatore, spesso separandosi in gruppi diversi ma aggregandosi regolarmente. Gli attacchi del predatore sono chiaramente visibili come massimi di velocità e distanza medie.

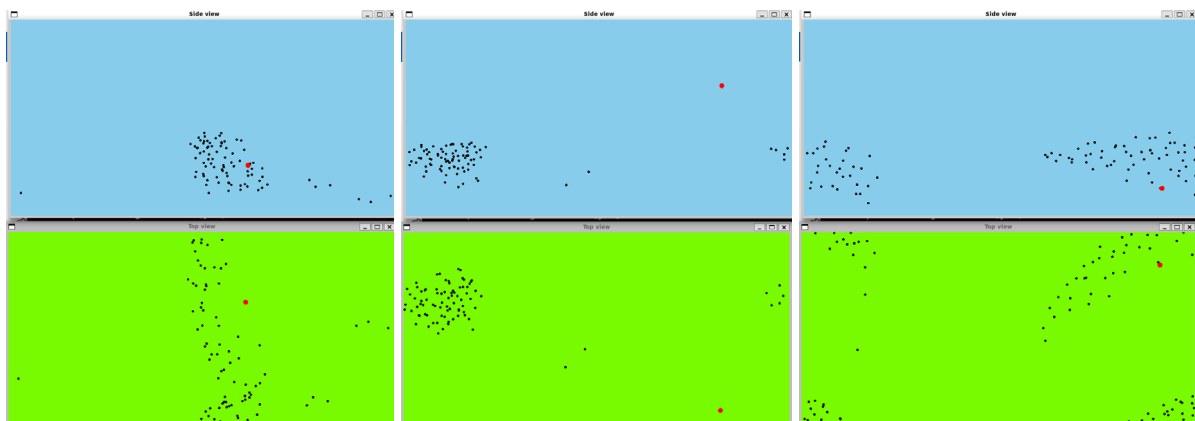


Figura 3: screenshot della simulazione 2

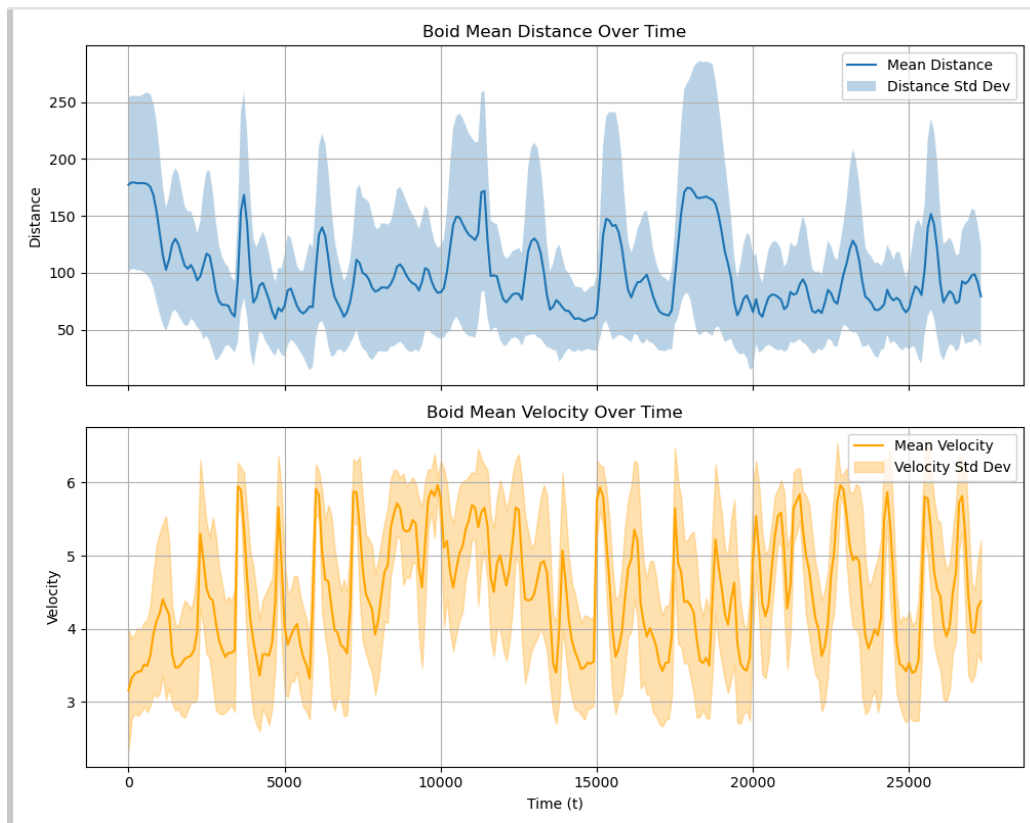


Figura 4: statistiche della simulazione 2

4.6.3 Test 3: Simulazione con vento e spazio toroidale

Input:

- Spazio toroidale: Presente
- Vento: Presente
- Numero di boid: 40
- Range di attacco: 60 m
- Velocità di attacco: 8 m/s
- Parametri di separazione, allineamento, coesione, paura e altezza: 20%, 100%, 40%, 50%, 100%

Output:

Stormo che si muove diradato con traiettoria fortemente influenzata dal vento, che porta i volatili a raggiungere presto la velocità massima, alcuni boid sono catturati ma lo stormo reagisce lentamente alla presenza del predatore. A causa del vento le velocità dei boid diventano uniformi in breve tempo.

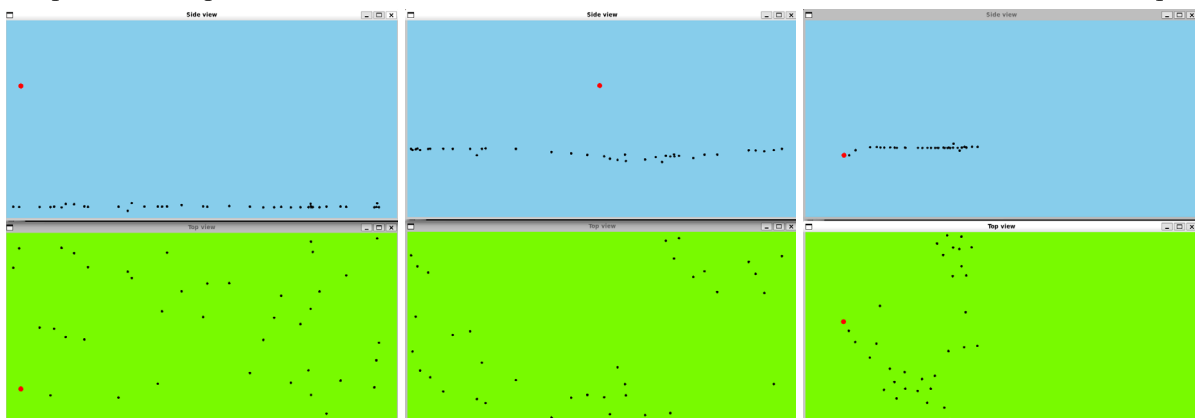


Figura 5: Screenshot delle simulazione 3

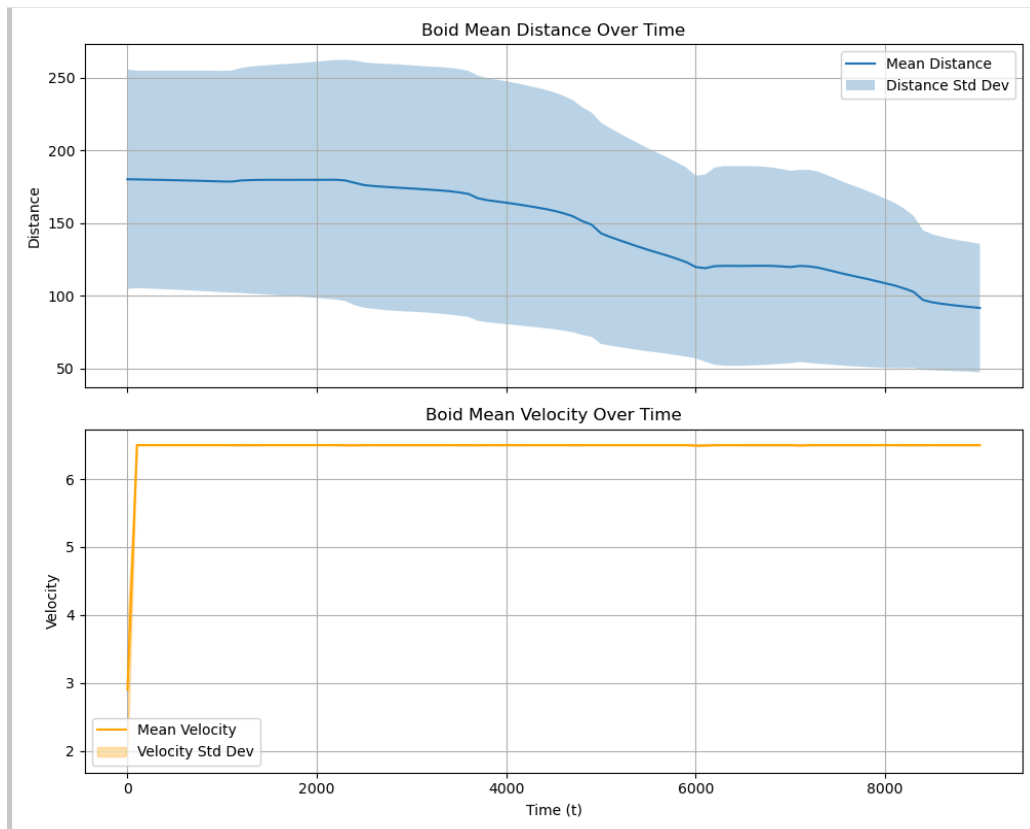


Figura 6: statistiche della simulazione 3

4.6.4 Test 4: Simulazione senza vento e con spazio toroidale

Input:

- Spazio toroidale: Presente
- Vento: Assente
- Predatore: Assente
- Numero di boid: 70
- Parametri di separazione, allineamento, coesione, paura e altezza: 50%, 50%, 50%, 50%, 50%

Output:

lo stormo si muove lentamente ma in modo coordinato. Dopo una fase di aggregazione iniziale, i volatili iniziano a muoversi in modo ordinato e con direzione fissa, effetto dovuto anche al campo visivo ridotto e rappresentato da una stabilizzazione delle distanze e velocità medie.

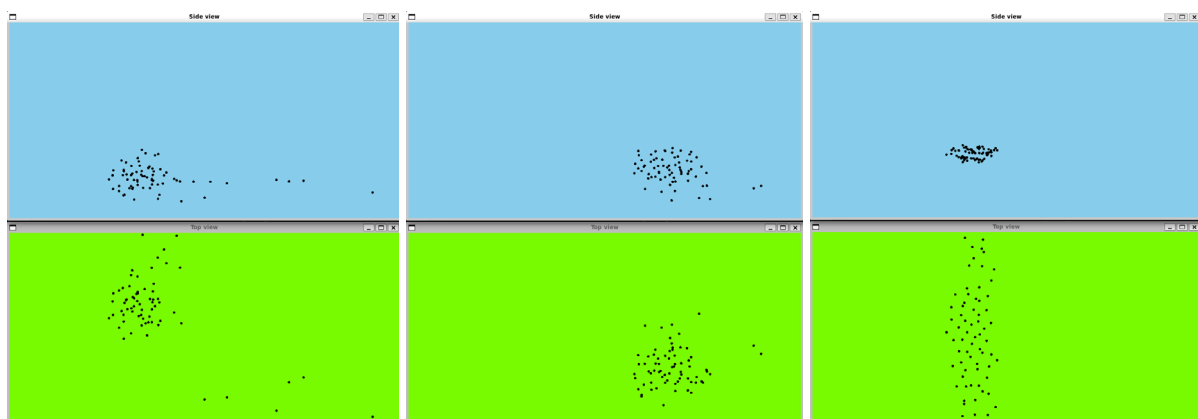


Figura 7: Screenshot della simulazione 4

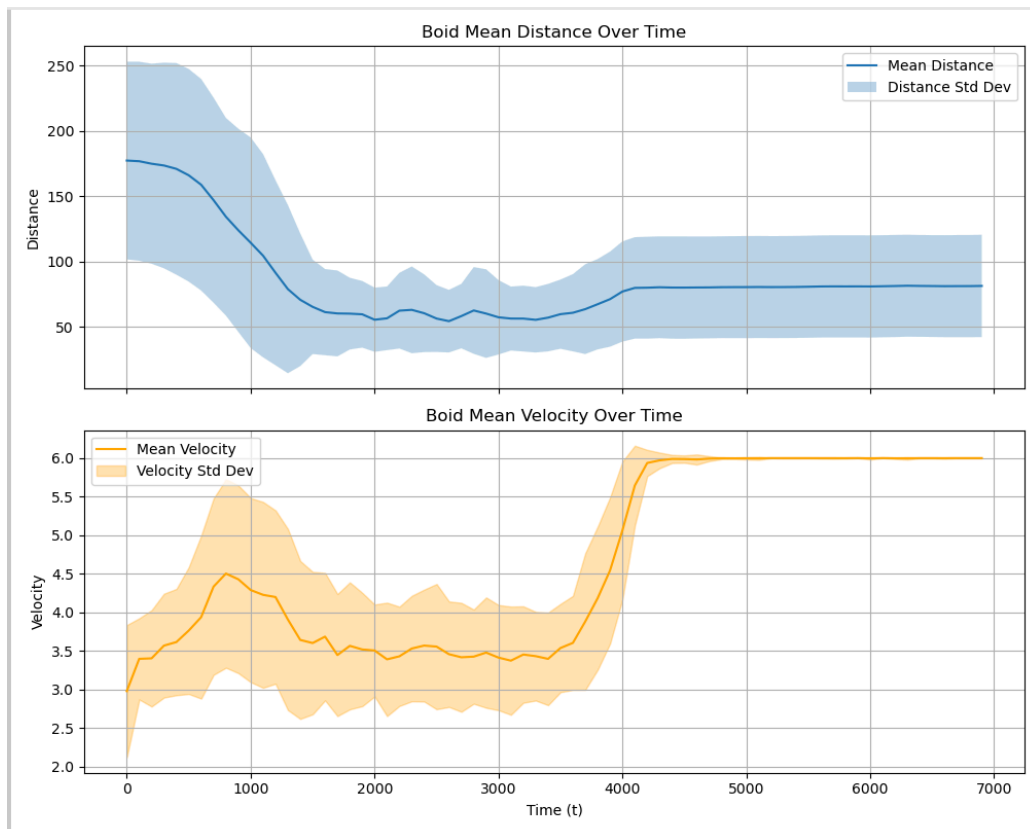


Figura 8: statistiche della simulazione 4

Conclusioni

La simulazione si svolge fluidamente con ogni combinazione di variabili provata, inclusi i test con parametri casuali. Per valutare l'aderenza del modello alla realtà non sono state considerate situazioni poco significative a livello fisico, come la presenza contemporanea di vento e bordi solidi. Lo stormo si comporta come previsto, mostrando una tendenza più o meno veloce ad aggregarsi e ad allineare le proprie velocità, sia in presenza di vento che non, e in entrambe le possibili configurazioni dello spazio, toroidale e non. Inoltre, le azioni del predatore sono facilmente deducibili dalle statistiche dello stormo.

Bibliografia

- [1] C. W. Reynolds. Flocks, Herds, and Schools: A Distributed Behavioral Model. Computer Graphics: SIGGRAPH '87 Conference Proceedings, 21(4):25–34, 1987.
- [2] <https://vergenet.net/~conrad/boids/pseudocode.html>

Appendice

- [3] Repository github contenente il progetto:
<https://github.com/Ciccio-Fonsy/Boids>